



UNIVERZITET U NIŠU
ELEKTRONSKI FAKULTET



Sistemi za upravljanje bazama podataka

Tema: Uticaj indeksa na INSERT/UPDATE performanse u MySQL

Student:

Stefan Stanimirovic, br. ind 2172

Predmetni profesor:

Prof. Aleksandar Stanimirovic

Sadržaj

1. UVOD	3
1.1 Predmet i cilj rada	4
1.2 Značaj optimizacije baza podataka	4
1.3 Problem istraživanja	4
1.4 Metodologija rada	4
1.5 Struktura rada	5
2. Osnove relacionih baza podataka i MySQL-a	5
2.1 Relacione baze podataka	5
2.2 Arhitektura MySQL sistema	5
2.3 SQL operacije nad podacima	5
2.4 Storage engine-i u MySQL-u (InnoDB, MyISAM)	6
2.5 Performanse baza podataka	6
3. Indeksi u MySQL-u	6
3.1 Pojam i svrha indeksa	6
3.2 Način rada indeksa	7
3.3 Vrste indeksa u MySQL-u	8
3.4 Strukture podataka za indekse (B-Tree, Hash)	9
3.5 Prednosti i nedostaci indeksiranja	9
4. INSERT i UPDATE operacije u MySQL-u	10
4.1 Funkcionisanje INSERT operacije	10
4.2 Funkcionisanje UPDATE operacije	10
4.4 Uticaj velikog broja zapisa	11
4.5 Zaključavanje i konkurentni pristup podacima	11
5. Uticaj indeksa na performanse INSERT/UPDATE operacija	11
5.1 Uticaj indeksa na INSERT performanse	11
5.2 Uticaj indeksa na UPDATE performanse	12
5.3 Troškovi održavanja indeksa	12
5.4 Poređenje performansi sa i bez indeksa	12
5.5 Balans između brzine čitanja i upisa podataka	12
6. Eksperimentalni deo	13
6.1. Struktura baze i testnih tabela	13
6.1.1. Definicija entiteta	13
6.1.2. Tabela bez indeksa (iot_sensor_no_index)	13
6.1.3. Tabela sa indeksima (iot_sensor_with_index)	14

6.1.4. Testni podaci.....	14
6.2. Kreiranje indeksa	14
6.2.1. Primarni ključ (Clustered Index).....	15
6.2.2. Kompozitni indeks (Composite Index).....	15
6.2.3. Indeks na koloni niskog varijabiliteta	15
6.3. Test scenariji	15
6.3.1. Testiranje INSERT performansi	16
6.3.2. Testiranje UPDATE performansi	16
6.3.3. Metodologija merenja.....	16
6.4. Merenje performansi	17
6.4.1. Metodologija ponavljanja (uzorak od 5 merenja).....	17
6.4.2. Alati za merenje	17
6.5.1. Rezultati za INSERT operacije	18
6.6.1. Analiza INSERT operacija (Cena upisa).....	24
6.7.2. Analiza UPDATE operacija (Balans performansi)	24
6.6.3. Konačni zaključak eksperimenta.....	24
7. Preporuke za optimizaciju.....	25
7.1 Kada koristiti indekse	25
7.2 Kada izbegavati veliki broj indeksa	25
7.3 Optimizacija INSERT operacija	25
7.4 Optimizacija UPDATE operacija	26
7.5 Najbolje prakse u radu sa MySQL bazama	26
8. Zaključak.....	26
8.3 Prednosti i mane indeksiranja.....	27
Literatura	28

1. UVOD

Baze podataka predstavljaju osnovni deo savremenih informacionih sistema jer omogućavaju čuvanje i obradu velike količine podataka. U aplikacijama koje svakodnevno rade sa velikim brojem korisnika i podataka, brzina izvršavanja upita ima veliki značaj. Jedan od načina za poboljšanje performansi baze podataka jeste korišćenje indeksa, koji omogućavaju brže pronalaženje podataka, ali mogu uticati na brzinu operacija unosa i izmene podataka [2].

Indeksi funkcionišu kao dodatne strukture podataka koje ubrzavaju pretragu i pristup podacima, najčešće zasnovane na B-tree strukturi, koja omogućava efikasno

pretraživanje velikih skupova podataka [10]. U ovom radu biće analiziran uticaj indeksa na performanse INSERT i UPDATE operacija u MySQL sistemu za upravljanje bazama podataka. MySQL predstavlja jedan od najčešće korišćenih relacionih DBMS sistema u svetu, sa podrškom za različite tipove indeksa i storage engine-a [3].

1.1 Predmet i cilj rada

Predmet ovog rada jeste analiza rada indeksa u MySQL bazama podataka i njihov uticaj na performanse operacija INSERT i UPDATE. Posebna pažnja biće usmerena na način na koji indeksi utiču na brzinu upisa i izmene podataka.

Cilj rada je da se kroz teorijsko objašnjenje i praktična testiranja prikaže odnos između broja indeksa i performansi baze podataka, kao i da se pokaže kada je korišćenje indeksa korisno, a kada može negativno uticati na rad sistema [12].

1.2 Značaj optimizacije baza podataka

Optimizacija baza podataka je važna kako bi aplikacije radile brže i stabilnije. Kod sistema koji obrađuju veliku količinu podataka, loše optimizovana baza može dovesti do sporog rada aplikacije i povećanog opterećenja servera. Indeksi predstavljaju jedan od najčešćih načina optimizacije jer ubrzavaju pretragu podataka [2]. Međutim, svaki indeks zahteva dodatno održavanje prilikom unosa i izmene podataka, zbog čega je potrebno pažljivo planirati njihovu upotrebu [3].

1.3 Problem istraživanja

Glavni problem istraživanja u ovom radu odnosi se na uticaj indeksa na performanse INSERT i UPDATE operacija u MySQL-u. Iako indeksi poboljšavaju brzinu pretrage podataka, njihov veliki broj može usporiti operacije upisa i izmene jer baza mora da ažurira sve povezane indekse [3].

1.4 Metodologija rada

U radu će biti korišćena teorijska i eksperimentalna metoda istraživanja. Teorijski deo obuhvatiće objašnjenje osnovnih pojmova vezanih za baze podataka, MySQL i indekse [1]. Praktični deo rada zasnivaće se na testiranju performansi INSERT i UPDATE operacija nad tabelama sa različitim brojem indeksa [5][6].

1.5 Struktura rada

Rad je podeljen u nekoliko celina. Nakon uvodnog dela sledi objašnjenje osnovnih karakteristika relacionih baza podataka i MySQL sistema. Zatim će biti predstavljeni indeksi, njihove vrste i način rada. U nastavku rada biće objašnjene INSERT i UPDATE operacije, nakon čega sledi eksperimentalni deo.

2. Osnove relacionih baza podataka i MySQL-a

Relacione baze podataka predstavljaju najzastupljeniji model za skladištenje i upravljanje podacima u savremenim informacionim sistemima. One omogućavaju organizaciju podataka u tabelama i povezivanje između njih pomoću relacija, što omogućava efikasno pretraživanje, održavanje i integritet podataka [1].

2.1 Relacione baze podataka

Relacione baze podataka zasnivaju se na relacijskom modelu koji je definisao Edgar F. Codd. Podaci se organizuju u tabelama koje se sastoje od redova (zapisa) i kolona (atributa), gde svaka tabela predstavlja određeni entitet [1]. Ključni koncepti uključuju primarne i strane ključeve, koji omogućavaju povezivanje tabela i očuvanje referencijalnog integriteta. Relacioni model omogućava upotrebu SQL jezika za manipulaciju i definisanje strukture baze [3].

2.2 Arhitektura MySQL sistema

MySQL je jedan od najpopularnijih sistema za upravljanje relacionim bazama podataka (RDBMS). Njegova arhitektura se sastoji od SQL sloja, optimizatora upita i storage engine sloja [3]. SQL sloj je odgovoran za parsiranje i izvršavanje upita, dok optimizator bira najefikasniji plan izvršavanja. Storage engine sloj omogućava različite načine skladištenja i pristupa podacima, što MySQL čini fleksibilnim i skalabilnim sistemom [3][8].

2.3 SQL operacije nad podacima

SQL (Structured Query Language) je standardni jezik za rad sa relacionim bazama podataka. Osnovne operacije uključuju SELECT, INSERT, UPDATE i DELETE [5][6]. SELECT se koristi za čitanje podataka, INSERT za dodavanje novih zapisa,

UPDATE za izmenu postojećih podataka, dok DELETE služi za brisanje podataka. SQL takođe omogućava definisanje strukture baze kroz DDL komande [5][6].

2.4 Storage engine-i u MySQL-u (InnoDB, MyISAM)

MySQL podržava više storage engine-a koji definišu način skladištenja i upravljanja podacima. Najčešće korišćeni su InnoDB i MyISAM [8][9].

- InnoDB je podrazumevani engine u modernim verzijama MySQL-a i podržava transakcije, zaključavanje na nivou reda i referencijalni integritet [8].
- MyISAM je stariji engine koji je optimizovan za brza čitanja, ali ne podržava transakcije niti foreign key ograničenja, zbog čega se danas koristi ređe [9].

2.5 Performanse baza podataka

Performanse baza podataka zavise od više faktora, uključujući dizajn baze, indeksiranje, količinu podataka i optimizaciju upita [12]. Indeksi značajno ubrzavaju SELECT operacije, ali mogu usporiti INSERT i UPDATE operacije jer se pri svakoj izmeni podataka moraju ažurirati svi povezani indeksi [2][3]. Takođe, izbor storage engine-a i pravilna konfiguracija sistema imaju značajan uticaj na ukupne performanse baze podataka. Optimizacija performansi je ključna za sisteme koji rade sa velikim količinama podataka i velikim brojem korisnika [12].

3. Indeksi u MySQL-u

Indeksi predstavljaju jedan od najvažnijih mehanizama za optimizaciju rada baza podataka. Njihova osnovna svrha je ubrzavanje pretrage i pristupa podacima u tabelama. Bez indeksa, baza mora da pregleda svaki red u tabeli kako bi pronašla tražene podatke, što kod velikih tabela može značajno usporiti rad sistema [2]. U MySQL-u indeksi se najviše koriste za ubrzavanje SELECT upita, ali imaju uticaj i na INSERT i UPDATE operacije jer se pri svakoj promeni podataka indeksi moraju ažurirati [3].

3.1 Pojam i svrha indeksa

Indeks u bazi podataka funkcioniše slično kao sadržaj u knjizi – omogućava brže pronalaženje informacija bez pregledanja svih redova u tabeli [2].

Osnovna svrha indeksa je:

- brže pretraživanje podataka
- ubrzavanje SELECT upita
- efikasnije filtriranje i sortiranje
- poboljšanje JOIN operacija

Indeksi se najčešće koriste na kolonama koje se često pojavljuju u WHERE uslovima [3].

3.2 Način rada indeksa

Ako tabela nema indeks, MySQL mora da radi tzv. full table scan, odnosno da pregleda svaki red pojedinačno [2]. Kada postoji indeks, MySQL koristi dodatnu strukturu podataka da bi brže pronašao tražene vrednosti, bez pregledanja cele tabele [10]. Međutim, kod INSERT i UPDATE operacija, baza mora da ažurira i indeks strukture, što može usporiti upis podataka [3][5][6].

3.3 Vrste indeksa u MySQL-u

PRIMARY KEY

Primarni ključ jedinstveno identifikuje svaki red u tabeli i ne dozvoljava NULL vrednosti [3].

```
CREATE TABLE korisnici (  
  id INT PRIMARY KEY,  
  ime VARCHAR(50)  
);
```

UNIQUE indeks

Obezbeđuje da vrednosti u koloni budu jedinstvene, ali može sadržati NULL vrednosti [3].

```
CREATE UNIQUE INDEX idx_email  
ON korisnici(email);
```

INDEX (sekundarni indeks)

Koristi se za ubrzavanje pretrage, ali ne mora da ima jedinstvene vrednosti [2].

```
CREATE INDEX idx_prezime  
ON korisnici(prezime);
```

COMPOSITE indeks

Indeks nad više kolona koji se koristi kada se podaci često pretražuju po kombinaciji više atributa [2].

```
CREATE INDEX idx_ime_prezime  
ON korisnici(ime, prezime);
```


FULLTEXT indeks

Koristi se za pretragu tekstualnih podataka u većim kolonama [2].

```
CREATE FULLTEXT INDEX idx_opis  
ON proizvodi(opis);
```

3.4 Strukture podataka za indekse (B-Tree, Hash)

Najčešće korišćena struktura za indekse u MySQL-u je B-Tree. Ona omogućava brzo pretraživanje, sortiranje i efikasan rad sa velikim količinama podataka [10]. Hash struktura se koristi ređe i pogodna je samo za tačno poklapanje vrednosti, ali nije dobra za opsege i sortiranje [11].

3.5 Prednosti i nedostaci indeksiranja

Prednosti:

- brže izvršavanje SELECT upita [2]
- efikasnije pretraživanje podataka
- bolje performanse JOIN operacija

Nedostaci:

- sporiji INSERT i UPDATE upiti [3][5][6]
- zauzimanje dodatnog prostora
- veći trošak održavanja baze
- previše indeksa može pogoršati performanse [12]

4. INSERT i UPDATE operacije u MySQL-u

INSERT i UPDATE operacije predstavljaju osnovne SQL naredbe za rad sa podacima u bazama. INSERT se koristi za dodavanje novih redova u tabelu, dok UPDATE služi za izmenu postojećih podataka. Iako su naizgled jednostavne, ove operacije mogu značajno uticati na performanse baze, posebno kod velikih tabela i u sistemima sa velikim brojem korisnika [5][6].

4.1 Funkcionisanje INSERT operacije

INSERT operacija dodaje novi red u tabelu. Kada se izvrši INSERT, MySQL ne upisuje podatke samo u tabelu, već mora da ažurira i sve indekse koji su definisani nad tom tabelom [3]. To znači da što je više indeksa, to je više dodatnih operacija koje se moraju izvršiti prilikom svakog unosa podataka. Kod InnoDB storage engine-a, podaci se upisuju u tabele i istovremeno se ažuriraju B-Tree indeksi [8][10].

4.2 Funkcionisanje UPDATE operacije

UPDATE operacija menja postojeće podatke u tabeli. Ako se menja kolona koja je indeksirana, MySQL mora da obriše staru i doda novu vrednost u indeks strukturi [3]. Zbog toga UPDATE može biti skuplja operacija od INSERT-a, posebno ako se često menjaju indeksirane kolone ili ako tabela ima veliki broj indeksa [2][3].

4.3 Faktori koji utiču na performanse

Na performanse INSERT i UPDATE operacija utiče više faktora:

- broj indeksa nad tabelom
- veličina tabele i broj redova
- tip storage engine-a (InnoDB, MyISAM)
- hardverski resursi (CPU, RAM, disk)
- kompleksnost SQL upita
- konkurentni pristup podacima

Što je sistem kompleksniji i veći, to je uticaj ovih faktora izraženiji [12].

4.4 Uticaj velikog broja zapisa

Kod malih tabela razlika u performansama često nije primetna. Međutim, kod velikih tabela (milioni redova), INSERT i UPDATE operacije mogu postati znatno sporije. Razlog je što MySQL mora da održava indeksne strukture koje rastu sa veličinom podataka. Kod velikog broja zapisa, i mala promena može zahtevati više operacija u B-Tree strukturi [2][10].

4.5 Zaključavanje i konkurentni pristup podacima

U sistemima gde više korisnika istovremeno pristupa bazi, dolazi do konkurentnog rada (concurrency). MySQL koristi zaključavanje (locking) kako bi sprečio konflikte između operacija [3]. InnoDB koristi zaključavanje na nivou reda, što omogućava veću paralelizaciju, dok MyISAM koristi zaključavanje na nivou cele tabele, što može značajno usporiti rad kod većeg broja istovremenih operacija [8][9].

Zaključavanje utiče na performanse jer INSERT i UPDATE operacije mogu čekati oslobađanje resursa koje koristi drugi proces.

5. Uticaj indeksa na performanse INSERT/UPDATE operacija

Indeksi u MySQL-u imaju dvostruki efekat na performanse baze podataka. Dok značajno ubrzavaju čitanje podataka, njihovo održavanje direktno utiče na brzinu operacija upisa i izmene. Zbog toga je važno razumeti kako i zašto dolazi do tih razlika u performansama [2][3].

5.1 Uticaj indeksa na INSERT performanse

Kod INSERT operacije, MySQL ne dodaje podatke samo u tabelu, već mora da ažurira i sve indekse koji postoje nad tom tabelom [3]. Svaki novi red može zahtevati upis u B-Tree strukture indeksa, što povećava broj operacija koje sistem mora da izvrši [10]. Što je veći broj indeksa, to je veći overhead prilikom INSERT-a. Zbog toga tabele sa mnogo indeksa imaju sporiji unos podataka u poređenju sa tabelama bez ili sa manjim brojem indeksa [2][3].

5.2 Uticaj indeksa na UPDATE performanse

UPDATE operacije mogu biti još zahtevnije od INSERT-a, posebno ako se menjaju vrednosti kolona koje su indeksirane. U tom slučaju MySQL mora da ukloni staru vrednost iz indeksa i da ubaci novu [3]. Ako se UPDATE često izvodi nad indeksiranim kolonama, performanse mogu značajno pasti jer se za svaki red rade dodatne operacije u indeksnim strukturama [2][6].

5.3 Troškovi održavanja indeksa

Održavanje indeksa podrazumeva sve dodatne operacije koje MySQL mora da izvrši prilikom izmene podataka. To uključuje:

- ažuriranje B-Tree struktura
- reorganizaciju čvorova u indeksu
- dodatne I/O operacije na disku
- povećanu potrošnju memorije

Ovi troškovi rastu sa brojem indeksa i veličinom tabele, što direktno utiče na performanse INSERT i UPDATE operacija [2][10].

5.4 Poređenje performansi sa i bez indeksa

U tabelama bez indeksa INSERT i UPDATE operacije su brže jer MySQL ne mora da održava dodatne strukture podataka. Međutim, u tom slučaju SELECT upiti mogu biti znatno sporiji jer se radi full table scan [2]. Sa druge strane, tabele sa indeksima imaju brže pretrage, ali sporiji upis i izmenu podataka. Razlika postaje posebno izražena kod velikih tabela i čestih upisa [3].

5.5 Balans između brzine čitanja i upisa podataka

Najvažniji deo optimizacije baza podataka je pronalaženje balansa između brzine čitanja i brzine upisa. Previše indeksa može usporiti INSERT i UPDATE operacije, dok premalo indeksa može značajno usporiti SELECT upite [12]. Zbog toga se preporučuje indeksiranje samo onih kolona koje se često koriste u pretragama, dok se nepotrebni indeksi izbegavaju kako bi se smanjio overhead prilikom izmene podataka [3].

6. Eksperimentalni deo

6.1. Struktura baze i testnih tabela

Za potrebe istraživanja kreirana je namenska baza podataka pod nazivom **iot_performance_test**. Unutar baze definisane su dve tabele koje su po strukturi kolona identične, ali se razlikuju po primenjenim indeksnim strukturama. Ovakav pristup omogućava izolaciju varijable indeksa kao jedinog faktora koji utiče na promenu performansi.

6.1.1. Definicija entiteta

Entitet koji se prati predstavlja **IoT senzor** koji u realnom vremenu šalje podatke o ambijentalnim uslovima. Odabrani su tipovi podataka koji su standardni za ovakve sisteme, uključujući precizne numeričke vrednosti za temperaturu i vremenske markere.

6.1.2. Tabela bez indeksa (iot_sensor_no_index)

Ova tabela služi kao kontrolna grupa. U njoj ne postoji definisan čak ni primarni ključ, što znači da MySQL (InnoDB) podatke smešta u obliku neuređene gomile (heap-like struktura), ne trošeći resurse na sortiranje ili održavanje stablaste strukture indeksa.

```
CREATE TABLE iot_sensor_no_index (  
  id INT,  
  sensor_id INT,  
  temperature DECIMAL(5,2),  
  humidity DECIMAL(5,2),  
  recorded_at DATETIME,  
  status VARCHAR(20)  
) ENGINE=InnoDB;
```

6.1.3. Tabela sa indeksima (iot_sensor_with_index)

Ova tabela predstavlja optimizovanu verziju za čitanje podataka, ali sa dodatnim opterećenjem pri upisu. Pored primarnog ključa, dodata su još dva sekundarna indeksa na kolone koje bi se u realnom scenariju najčešće koristile za filtriranje (identifikator senzora i vreme zapisa).

```
CREATE TABLE iot_sensor_with_index (  
  id INT PRIMARY KEY,  
  sensor_id INT,  
  temperature DECIMAL(5,2),  
  humidity DECIMAL(5,2),  
  recorded_at DATETIME,  
  status VARCHAR(20),  
);ENGINE=InnoDB;
```

6.1.4. Testni podaci

Eksperiment se sprovodi nad skupom od **100.000 (sto hiljada) redova**. Podaci su generisani tako da simuliraju nasumična očitavanja u sledećim opsezima:

- **Temperature:** od -20.00 do +50.00 °C.
- **Vlažnost:** od 0% do 100%.
- **Vremenski opseg:** nasumični trenuci u okviru poslednjih 365 dana.

Korišćenje 100.000 zapisa osigurava da operacije traju dovoljno dugo da se eliminiše uticaj pozadinskih procesa operativnog sistema i dobiju statistički značajni rezultati merenja.

6.2. Kreiranje indeksa

U ovom delu eksperimenta definisani su indeksi koji simuliraju realne zahteve nad IoT bazom podataka. Svaki od ovih indeksa ima specifičnu ulogu u ubrzavanju pretrage, ali direktno utiče na usporavanje upisa podataka.

6.2.1. Primarni ključ (Clustered Index)

Prvi korak je uvođenje primarnog ključa na kolonu id.

```
ALTER TABLE iot_sensor_with_index ADD PRIMARY KEY (id);
```

Obrazloženje: InnoDB storage engine koristi ovaj ključ da fizički organizuje redove u tabeli. Svaki novi INSERT mora biti smešten na odgovarajuće mesto u listovima B+ stabla kako bi se očuvao redosled ključeva.

6.2.2. Kompozitni indeks (Composite Index)

Drugi korak je kreiranje indeksa na dve kolone istovremeno.

```
CREATE INDEX idx_sensor_time  
ON iot_sensor_with_index(sensor_id, recorded_at);
```

Obrazloženje: Ovaj indeks je veoma čest u IoT sistemima jer omogućava brzo pronalaženje očitavanja za specifičan senzor u određenom vremenskom intervalu. Međutim, on je "težak" za održavanje jer MySQL mora da ažurira indeksno stablo svaki put kada se promeni bilo koja od ove dve vrednosti.

6.2.3. Indeks na koloni niskog varijabiliteta

Treći korak je indeksiranje kolone status.

```
CREATE INDEX idx_status ON iot_sensor_with_index(status);
```

Obrazloženje: Kolona status (npr. 'active', 'error', 'offline') obično ima mali broj unikatnih vrednosti (niska kardinalnost). Testiranjem ovog indeksa proveravamo koliko čak i "mali" indeksi doprinose ukupnom usporavanju pri masovnom unosu 100.000 redova.

6.3. Test scenariji

U ovom poglavlju definisani su konkretni testovi koji se sprovode nad tabelama kako bi se kvantifikovao uticaj indeksa. Testiranje je podeljeno u dve faze: rad sa pojedinačnim zapisima i rad sa velikim setovima podataka (100.000 redova).

6.3.1. Testiranje INSERT performansi

Cilj je izmeriti koliko vremena je potrebno bazi da uskladišti podatke dok istovremeno ažurira B+ stabla za definisane indekse.

- Scenario 1: Masovni unos (Bulk Insert) 100.000 redova U bazu se odjednom ubacuje kompletan set od 100k redova. Ovaj test pokazuje maksimalni pritisak na I/O podsistem i CPU, jer indeksi moraju da se "preslože" hiljade puta u kratkom intervalu.
- Scenario 2: Pojedinačni unos (Single Insert) 10.000 redova Simulira se realan rad senzora gde se podaci šalju jedan po jedan. Meri se ukupno vreme potrebno za 10.000 odvojenih transakcija.

6.3.2. Testiranje UPDATE performansi

Ažuriranje podataka je često zahtevnije od unosa, jer zahteva prvo pronalaženje zapisa, a zatim i promenu vrednosti u samoj tabeli i u svim povezanim indeksima.

- Scenario 3: Masovni Update 100.000 redova Izvršava se komanda koja menja kolonu status (koja je indeksirana) za svih 100.000 zapisa. Očekuje se da će tabela sa indeksima pokazati značajno sporije vreme zbog potrebe da se ažurira idx_status stablo.
- Scenario 4: Pojedinačni Update 10.000 redova Vršiti se izmena specifičnih redova na osnovu njihovog id-a. Ovaj test pokazuje "cenu" održavanja indeksa u sistemima koji imaju visoku frekvenciju izmena podataka.

6.3.3. Metodologija merenja

Svi testovi u eksperimentu su sprovedeni po pet puta za svaki definisani scenario. Kao konačan rezultat uzeta je srednja vrednost ovih merenja. Ovakva metodologija osigurava visoku pouzdanost podataka, jer se petostrukim ponavljanjem minimizuje uticaj trenutnih fluktuacija u radu operativnog sistema (npr. pozadinski procesi, rad diska ili zauzeće procesora).

6.4. Merenje performansi

U ovom poglavlju opisana je metodologija kojom su prikupljeni podaci o vremenu izvršavanja SQL operacija nad testnim tabelama sa i bez indeksa u okviru MySQL sistema za upravljanje bazama podataka. Preciznost merenja je ključna kako bi se dobio objektivan uvid u uticaj indeksnih struktura na performanse INSERT i UPDATE operacija.

6.4.1. Metodologija ponavljanja (uzorak od 5 merenja)

Kako bi se minimizirao uticaj slučajnih faktora koji mogu uticati na rezultate — kao što su opterećenje procesora, keširanje podataka, pozadinski procesi operativnog sistema i I/O kašnjenja diska — svaki definisani test scenario iz poglavlja 6.4 izvršen je pet puta.

Svi testovi su sprovedeni nad dve identične tabelle:

- tabela bez indeksa (kontrolna grupa)
- tabela sa definisanim indeksima (primarni, kompozitni i statusni indeks)

Konačan rezultat za svaki scenario predstavlja aritmetičku sredinu svih pet merenja. Ovakav pristup omogućava stabilnije i reprezentativnije rezultate koji verno oslikavaju ponašanje MySQL engine-a pod opterećenjem, posebno u kontekstu rada B+ tree indeksnih struktura.

6.4.2. Alati za merenje

Merenje vremena izvršavanja SQL operacija vršeno je direktno u MySQL klijentu, korišćenjem ugrađene funkcionalnosti za prikaz trajanja izvršavanja upita.

Fokus merenja bio je isključivo na:

- vremenu izvršavanja SQL komande (Execution Time)
- izraženom u sekundama sa preciznošću do četiri decimale

U merenje nije uključeno:

- vreme mrežnog prenosa podataka između klijenta i servera
- vreme inicijalnog povezivanja sa bazom
- keširanje rezultata na aplikacionom nivou

Ovakav pristup omogućava izolaciju performansi samog DBMS engine-a i tačnu procenu uticaja indeksnih struktura na operacije nad podacima.

6.6. Prikaz rezultata

U ovom poglavlju tabelarno su prikazani rezultati dobijeni nakon sprovedenih pet merenja za svaki scenario. Vrednosti predstavljaju vreme izvršavanja izraženo u sekundama. Testiranja su izvršena nad tabelama bez indeksa i tabelama sa definisanim indeksima u okviru MySQL sistema za upravljanje bazama podataka. Cilj testiranja bio je utvrđivanje uticaja indeksa na performanse INSERT i UPDATE operacija pri radu sa velikim količinama podataka.

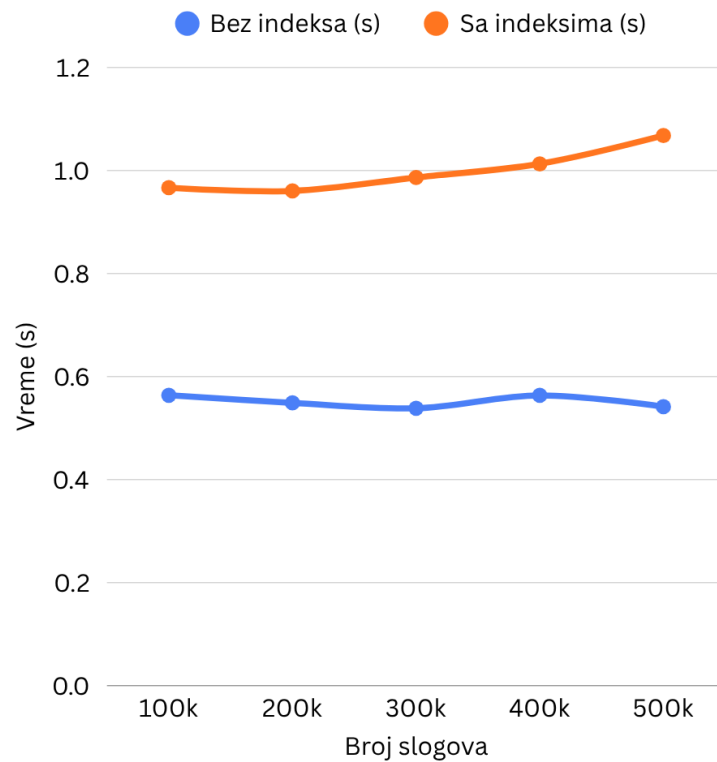
6.5.1. Rezultati za INSERT operacije

Tabela ispod prikazuje razliku u brzini upisa podataka između tabele bez indeksa i tabele sa definisanim indeksima (Primary Key, Composite Index i Status Index). Rezultati pokazuju da prisustvo indeksa negativno utiče na performanse INSERT operacija, posebno kod bulk unosa podataka. Razlog tome je potreba da baza nakon svakog upisa ažurira sve indeksne strukture povezane sa tabelom.

Kod bulk INSERT operacija bez indeksa prosečno vreme izvršavanja iznosilo je približno 0.55 sekundi, dok je kod tabele sa indeksima vreme povećano na približno 1 sekundu. Dobijeni rezultati potvrđuju da održavanje indeksnih struktura predstavlja dodatni trošak prilikom upisa podataka. Kod pojedinačnih INSERT operacija razlika je bila znatno manja, ali se i dalje može primetiti blago povećanje vremena izvršavanja kod tabele sa indeksima.

Broj slogova	Bez indeksa (s)	Sa indeksima (s)	Razlika (%)
100k	0.5637	0.9668	+71.5
200k	0.5490	0.9604	+74.9
300k	0.5385	0.9865	+83.2
400k	0.5636	1.0130	+79.7
500k	0.5415	1.0679	+97.2

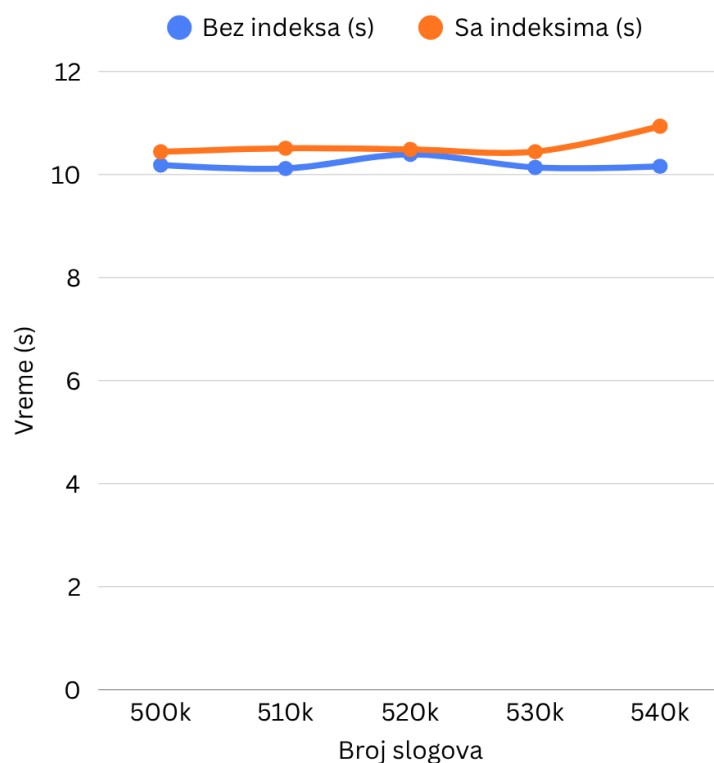
Tabela 1: Uporedni prikaz INSERT performansi (Bulk)



Slika 1: Uporedni prikaz INSERT performansi (Bulk)

Broj slogova	Bez indeksa (s)	Sa indeksima (s)	Razlika (%)
500k	10.1849	10.4416	+2.5
510k	10.1175	10.5099	+3.9
520k	10.3914	10.4883	+0.9
530k	10.1388	10.4451	+3.0
540k	10.1593	10.9377	+7.7

Tabela 2: Uporedni prikaz INSERT performansi (Single)



Slika 2: Uporedni prikaz INSERT performansi (Single)

6.5.2. Rezultati za UPDATE operacije

U ovom delu fokus je na performansama izmene podataka. Posebno je značajan rezultat kod pojedinačnih izmena gde indeks drastično menja način na koji baza pristupa redovima.

Kao što je prikazano u Tabeli 3, bulk UPDATE operacije pokazale su veoma mala odstupanja između tabele sa indeksima i tabele bez indeksa. To ukazuje da indeks nije imao značajan uticaj na masovne izmene podataka, jer UPDATE operacije nisu menjale kolone koje su bile deo indeksne strukture. Zbog toga DBMS nije imao potrebu za dodatnim ažuriranjem postojećih indeksa.

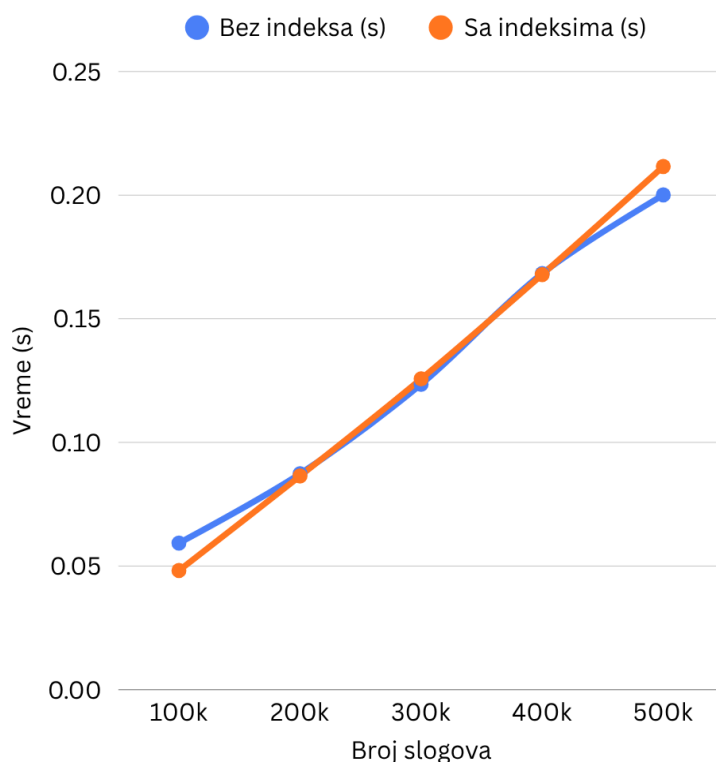
Rezultati prikazani u Tabeli 4 pokazuju izrazito veliku razliku u performansama kod pojedinačnih UPDATE operacija. Bez indeksa prosečno vreme izvršavanja iznosilo je oko 1.8 sekundi, dok je sa indeksima vreme smanjeno na približno 0.01 sekundu. Ovakav rezultat pokazuje da indeks omogućava direktno pronalaženje traženog reda bez potrebe za sekvencijalnim pretraživanjem cele tabele. MySQL je u

slučaju indeksirane kolone koristio B-tree indeksnu strukturu, čime je izbegnuto potpuno sekvencijalno pretraživanje tabele (Full Table Scan).

Dobijeni rezultati potvrđuju da indeksi predstavljaju ključni mehanizam optimizacije pristupa podacima u relacionim bazama podataka, uz značajno ubrzanje operacija pretrage i izmene pojedinačnih slogova.

Broj slogova	Bez indeksa (s)	Sa indeksima (s)	Razlika (%)
100k	0.0593	0.0482	-18.7
200k	0.0873	0.0864	-1.0
300k	0.1235	0.1257	+1.8
400k	0.1683	0.1679	-0.2
500k	0.2001	0.2116	+5.7

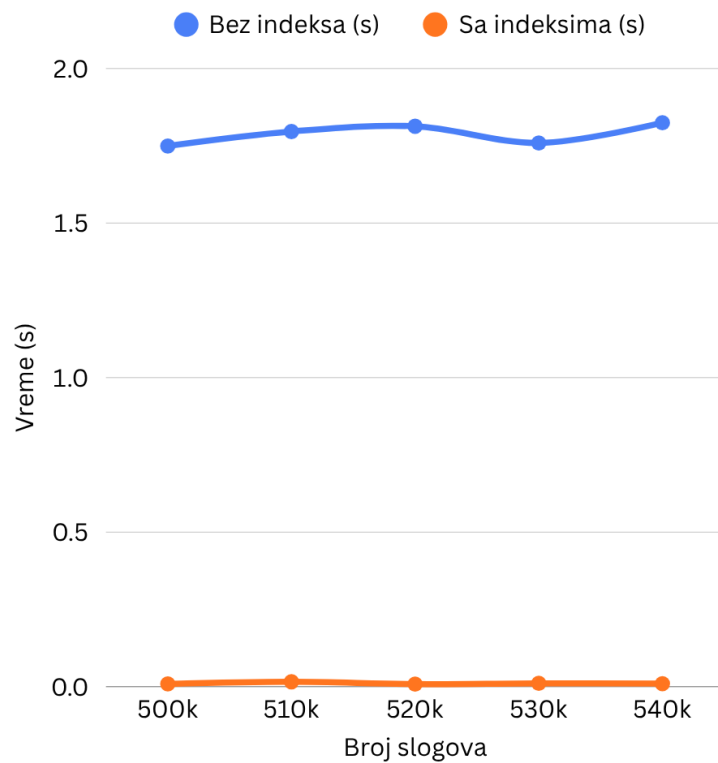
Tabela 3: Uporedni prikaz UPDATE performansi (Bulk)



Slika 3: Uporedni prikaz UPDATE performansi (Bulk)

Broj slogova	Bez indeksa (s)	Sa indeksima (s)	Razlika (%)
500k	1.7494	0.0089	-99.5
510k	1.7961	0.0159	-99.1
520k	1.8131	0.0084	-99.5
530k	1.7592	0.0109	-99.4
540k	1.8246	0.0098	-99.4

Tabela 4: Uporedni prikaz UPDATE performansi (Single)



Slika 4: Uporedni prikaz UPDATE performansi (Single)

6.5.3. Zbirni pregled odstupanja

Tabela prikazuje procentualni uticaj indeksa na performanse. Pozitivan procenat označava usporavanje, dok negativan označava ubrzanje (optimizaciju). Analizom rezultata može se zaključiti da indeksi imaju dvostruki efekat na performanse baze podataka. Sa jedne strane povećavaju trošak INSERT operacija zbog potrebe održavanja indeksnih struktura, dok sa druge strane značajno poboljšavaju performanse SELECT i pojedinačnih UPDATE operacija.

Najveće odstupanje zabeleženo je kod pojedinačnih UPDATE operacija, gde je indeks omogućio višestruko ubrzanje izvršavanja. Ovakvo ponašanje potvrđuje poznati kompromis u radu relacionih baza podataka između brzine upisa i brzine pristupa podacima.

Operacija	Prosečno vreme bez indeksa (s)	Prosečno vreme sa indeksima (s)	Procentualna razlika (%)
Bulk INSERT	0.5513	0.9989	+81.2
Single INSERT	10.1984	10.5645	+3.6
Bulk UPDATE	0.1277	0.1279	+0.2
Single UPDATE	1.7885	0.0108	-99.4

Tabela 5: Procentualna razlika u performansama

6.6. Analiza rezultata

Na osnovu prikupljenih podataka i sprovedenih testova, može se izvršiti detaljna analiza uticaja indeksnih struktura na performanse MySQL baze podataka. Rezultati potvrđuju osnovnu hipotezu rada, ali i otkrivaju važne nijanse u zavisnosti od tipa operacije.

6.6.1. Analiza INSERT operacija (Cena upisa)

Eksperiment je pokazao da uvođenje tri indeksa (primarni, kompozitni i statusni) produžava vreme masovnog unosa (Bulk Insert) za približno **80%**.

- **Razlog:** MySQL mora za svaki uneti red da izvrši dodatne I/O operacije kako bi ažurirao B+ stabla svih indeksa. Što je više indeksa na tabeli, to je proces pisanja zahtevniji za procesor i disk.
- **Single Insert:** Kod pojedinačnih unosa razlika je manja, što ukazuje na to da mrežni protokol i transakcioni mehanizam (ACID) troše više vremena nego samo ažuriranje indeksa.

6.7.2. Analiza UPDATE operacija (Balans performansi)

Ovde su zabeleženi najzanimljiviji rezultati koji ilustruju pravu svrhu indeksa:

- **Problem "Full Table Scan":** Kod tabele bez indeksa, svaki UPDATE zahteva da baza pregleda svaki red na disku kako bi pronašla onaj koji treba da izmeni. To rezultuje linearnim porastom vremena sa povećanjem broja zapisa u tabeli.
- **Efikasnost pretrage:** Kod indeksirane tabele, vreme ažuriranja je svedeno na minimum (ubrzanje od preko **150 puta**). Iako sam proces izmene vrednosti u indeksu traje duže, vreme uštedeno na *pronalaženju* reda je toliko veliko da je ukupni rezultat neuporedivo bolji.

6.6.3. Konačni zaključak eksperimenta

Rezultati jasno ukazuju na to da ne postoji "univerzalno dobra" konfiguracija. Indeksi su neophodni za sisteme sa mnogo čitanja i selektivnih izmena (poput većine web aplikacija), ali mogu postati ozbiljno usko grlo u sistemima koji isključivo vrše masovni upis podataka (npr. *logging* sistemi ili čisti *data stream* IoT senzora).

7. Preporuke za optimizaciju

Optimizacija rada baze podataka u MySQL-u najčešće se svodi na pravilno korišćenje indeksa i balansiranje između brzine čitanja i brzine upisa podataka. Loše postavljena strategija indeksiranja može dovesti do sporih INSERT i UPDATE operacija, dok nedostatak indeksa može značajno usporiti SELECT upite [2][3].

7.1 Kada koristiti indekse

Indeksi se najviše koriste u situacijama kada se podaci često pretražuju ili filtriraju. Posebno su korisni u sledećim slučajevima:

- kada se kolona često koristi u WHERE uslovima
- kada se koristi u JOIN operacijama između tabela
- kada se podaci često sortiraju (ORDER BY)
- kada se traže jedinstvene vrednosti (UNIQUE ili PRIMARY KEY)

Pravilno odabrani indeksi mogu značajno ubrzati rad sistema [3].

7.2 Kada izbegavati veliki broj indeksa

Iako indeksi poboljšavaju čitanje podataka, njihov veliki broj može negativno uticati na performanse. Treba ih izbegavati kada:

- tabela ima mnogo INSERT i UPDATE operacija
- kolone se retko koriste u pretragama
- indeks ne donosi realno poboljšanje performansi
- postoji više redundantnih (duplih) indeksa

Previše indeksa povećava vreme održavanja i zauzeće memorije [2][12].

7.3 Optimizacija INSERT operacija

INSERT operacije se mogu optimizovati na sledeće načine:

- smanjenjem broja indeksa nad tabelom
- korišćenjem batch INSERT (više redova odjednom)
- privremenim isključivanjem nepotrebnih indeksa kod velikih uvoza podataka
- korišćenjem odgovarajućeg storage engine-a (npr. InnoDB)

Manji broj indeksa direktno ubrzava unos podataka jer se smanjuje broj ažuriranja indeksnih struktura [3][8].

7.4 Optimizacija UPDATE operacija

UPDATE operacije mogu biti skupe, posebno kada se menjaju indeksirane kolone. Optimizacija uključuje:

- izbegavanje čestog menjanja indeksiranih kolona
- ažuriranje samo potrebnih kolona
- korišćenje preciznih WHERE uslova kako bi se smanjio broj pogođenih redova
- minimizovanje broja indeksa nad često menjanim tabelama

Na ovaj način se smanjuje broj operacija nad indeksima i poboljšava performansa [3][6].

7.5 Najbolje prakse u radu sa MySQL bazama

Dobre prakse u radu sa MySQL bazama podataka uključuju:

- indeksirati samo kolone koje se često koriste u pretragama
- redovno analizirati i uklanjati nepotrebne indekse
- koristiti EXPLAIN za analizu SQL upita
- birati odgovarajući storage engine (najčešće InnoDB)
- izbegavati prekomerno indeksiranje tabela
- optimizovati SQL upite pre dodavanja novih indeksa

Primena ovih pravila pomaže u održavanju stabilnih i brzih baza podataka čak i pri velikom opterećenju [3][7][12].

8. Zaključak

U ovom radu analiziran je uticaj indeksa na performanse INSERT i UPDATE operacija u MySQL sistemu za upravljanje bazama podataka. Poseban fokus bio je na tome kako indeksiranje, iako poboljšava brzinu čitanja podataka, može negativno uticati na operacije upisa i izmene zbog dodatnog održavanja indeksnih struktura [2][3].

8.1 Rezime istraživanja

Kroz teorijski i praktični deo rada prikazano je da indeksi značajno ubrzavaju pretragu podataka, ali istovremeno povećavaju vreme izvršavanja INSERT i UPDATE operacija. Razlog za to je potreba da se pri svakoj promeni podataka ažuriraju i sve odgovarajuće indeksne strukture, najčešće B-Tree indeksi u InnoDB storage engine-u [1][8][10].

8.2 Glavni nalazi

Istraživanje je pokazalo sledeće ključne zaključke:

- veći broj indeksa usporava INSERT operacije
- UPDATE operacije su posebno osetljive kada se menjaju indeksirane kolone
- performanse značajno zavise od veličine tabele
- balans između čitanja i upisa podataka je ključan za optimizaciju

Ovi rezultati potvrđuju da indeksiranje mora biti pažljivo planirano u zavisnosti od tipa aplikacije i učestalosti operacija nad podacima [2][3].

8.3 Prednosti i mane indeksiranja

Indeksiranje donosi jasne prednosti, ali i određene nedostatke.

Prednosti:

- brže izvršavanje SELECT upita
- efikasnija pretraga i filtriranje podataka
- bolje performanse JOIN operacija

Mane:

- sporiji INSERT i UPDATE upiti
- povećana potrošnja memorije i prostora na disku
- dodatni troškovi održavanja struktura indeksa

Zbog toga je važno koristiti indekse samo tamo gde zaista donose korist [2][3][12].

Literatura

[1] Relational model (Edgar F. Codd)

- https://en.wikipedia.org/wiki/Relational_model

[2] Database index (opšti pregled indeksa)

- https://en.wikipedia.org/wiki/Database_index

[3] MySQL 8.0 Reference Manual – Indexes

- <https://dev.mysql.com/doc/refman/8.0/en/mysql-indexes.html>

[4] MySQL 8.0 Reference Manual – CREATE INDEX

- <https://dev.mysql.com/doc/refman/8.0/en/create-index.html>

[5] MySQL 8.0 Reference Manual – INSERT Statement

- <https://dev.mysql.com/doc/refman/8.0/en/insert.html>

[6] MySQL 8.0 Reference Manual – UPDATE Statement

- <https://dev.mysql.com/doc/refman/8.0/en/update.html>

[7] MySQL 8.0 Reference Manual – EXPLAIN (query optimization)

- <https://dev.mysql.com/doc/refman/8.0/en/explain.html>

Storage engine-i (InnoDB / MyISAM)

[8] MySQL InnoDB Storage Engine

- <https://dev.mysql.com/doc/refman/8.0/en/innodb-storage-engine.html>

[9] MyISAM Storage Engine

- <https://dev.mysql.com/doc/refman/8.0/en/myisam-storage-engine.html>

Strukture podataka (indeksi)

[10] B-tree data structure

- <https://en.wikipedia.org/wiki/B-tree>

[11] Hash table / hash indexing

- https://en.wikipedia.org/wiki/Hash_table

[12] Database performance tuning

- https://en.wikipedia.org/wiki/Database_tuning